MARK:

DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY
REPORT TEMPLAT

Name	:	BRANDON LAMAN ANAK BEJI		**Registration No.**	:	20DIT24F1029
Code & Course	:	DFP40443 FULL STACK WEB DEVELOPMENT		**Class**	:	DIT4B
Lecturer	:	MOHD AZLAN BIN AB AZIZ		**Date**	:	20/4/2026
Assesment		PRACTICAL WORK 2				

Put your report inside this.

Database & Connection Setup	Constructs the database and products table flawlessly. Integrates db.php perfectly using a procedural mysqli connection and securely calibrates it with a conditional check to terminate execution if the connection fails	Constructs the database and connection script successfully, but error handling lacks a strict termination command or clear message.	Assembles a connection but mixes procedural and object-oriented styles, or completely omits the conditional error check	Attempts to write a database connection, but the script fails to connect to the MySQL server due to syntax or logical errors.

```php
<?php

error_reporting(E_ALL);
ini_set('display_errors', 1);

$servername = "localhost:3306";
$username = "root";
$password = "";
$dbname = "ecommerce_db";

$conn = mysqli_connect($servername, $username, $password, $dbname);
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
?>
```

Database connection is good since no error happened.

```php
include 'db.php';
```

this line is included in every file (view_products.php, add_product.php, delete_product.php, edit_product.php) so db.php is integrated to the code.

Form Processing & Validation	Builds a Bootstrap 5 form utilizing POST and enctype="multipart/form-data"3. Calibrates strict client-side (JS) and server-side (PHP) validation. Organizes graceful error handling by retaining user input (sticky forms)34	Builds the form and validation successfully, but misses one requirement (e.g., forgets JS validation or fails to make the form perfectly sticky on error).	Assembles a basic form but relies exclusively on HTML5 for validation without PHP server-side validation, or forgets the enctype attribute.	Fails to construct a functional form; data cannot be submitted, or input validation is entirely absent

```php
$pname = htmlspecialchars(stripslashes(trim($_POST['product_name'])));
$pprice = $_POST['price'];

$valid = true;

if (empty($pname)) { $nameErr = "Name is required"; $valid = false; }
if (!is_numeric($pprice) || $pprice <= 0) { $priceErr = "Valid price required"; $valid = false; }
```

Uses trim(),stripslashes(), and htmlspecialchars() to sanitize input so special character such as ; or ‘ will be romove

```
function validateForm() {
    let name = document.getElementById('pname').value;
    let price = document.getElementById('pprice').value;
    let image = document.getElementById('pimage').value;
    if (name == "" || price == "" || image == "" || price <= 0) {
        alert("All fields are required and price must be positive!")
        return false;
    }
    return true;
}
```

validateForm() is use to make sure no null, meet requirement in this case price can’t be 0

```
<div class>
    <label>Product Name</label>
    <input type="text" name="product_name" id="pname" class="form-control" value="<?php echo $pname; ?>">
    <span class="text-danger"><?php echo $nameErr; ?></span>
</div>
```

Sticky Form allow input to be retained even after error happen

File Upload & Management	Manipulates the $_FILES superglobal securely by verifying file size, type, and upload errors3. Executes move_uploaded_file() safely. Properly utilizes unlink() to remove old images during updates or deletions	Manipulates files correctly during the initial upload but fails to execute unlink() to remove old/deleted images from the server space during updates or deletions.	Assembles the file upload logic, but skips size and type verification, allowing any file format to be uploaded to the server.	Attempts to handle file uploads but fails; images are not saved to the designated directory.

```
if ($_FILES["image"]["size"] > 2000000) { $imageErr = "File too large (Max 2MB)"; $valid = false; }
if (!in_array($imageFileType, ["jpg", "jpeg", "png"])) { $imageErr = "Only JPG, JPEG, PNG allowed"; $valid = false; }
```

Verify the file type (Only allow JPG,JPEG,PNG) and size (only size that is less than 2MB)

```
if (move_uploaded_file($_FILES["image"]["tmp_name"], $target_file)) {
    $sql = "INSERT INTO products (product_name, price, image_path) VALUES (?, ?, ?)";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, "sds", $pname, $pprice, $target_file);
```

move_uploaded_file allow image to be move safely

```
if ($row) {
    if (file_exists($row['image_path'])) {
        unlink($row['image_path']);
    }
```

The usage of unlink on edit_product.php

```
if (file_exists($old_path) && !empty($old_path)) {
    unlink($old_path);
}
$final_path = $target_file;
```

The usage of unlink on update_product.php

unlink() is use to delete file in the server in this situation it will remove the image path

Read Data & Interface Design	Organizes the product catalog flawlessly in a responsive Bootstrap table. Executes a SELECT query and accurately renders product images and prices. Integrates dynamic 'Edit' and 'Delete' URLs passing the specific id	Organizes the dashboard successfully, but the Bootstrap styling is slightly broken, or images are stretched/misaligned due to missing CSS width constraints.	Assembles a raw HTML table to display data. Fails to integrate Bootstrap styling, or action buttons do not append the id to the query string properly.	Fails to retrieve data; the SELECT query contains errors, resulting in a blank page or SQL warnings.

```php
<?php
$sql = "SELECT * FROM products";
$result = mysqli_query($conn, $sql);
while($row = mysqli_fetch_assoc($result)) {
    echo "<tr>
        <td>{$row['id']}</td>
        <td><img src='{$row['image_path']}' width='50'></td>
        <td>{$row['product_name']}</td>
        <td>RM{$row['price']}</td>
        <td>
            <a href='edit_product.php?id={$row['id']}' class='btn btn-sm btn-success'>Edit</a>
            <a href='delete_product.php?id={$row['id']}' class='btn btn-sm btn-danger' onclick=\"return confirm('Are you sure?')\">Delete</a>
        </td>
    </tr>";
}
?>
```

Product Catalog

Add New Product

ID	Image	Name	Price	Actions
1		T-Shirt	RM17	Edit Delete
2		Dress	RM60	Edit Delete
3		Suit	RM289.99	Edit Delete

Update & Delete Operations	Manipulates records seamlessly. Builds an update form that perfectly pre-fills existing data. Integrates a JavaScript confirmation prompt before deletions. Executes both UPDATE and DELETE queries securely using Procedural Prepared Statements	Manipulates records well, but misses one key security or UX feature (e.g., fails to use Prepared Statements for the update, or forgets the JS confirmation prompt on delete)	Assembles update/delete scripts but uses raw un-sanitized $_GET variables directly inside standard queries, leaving the system vulnerable to SQL injection	Attempts to implement update/delete mechanisms, but the scripts crash when retrieving the ID, or the database records remain unchanged.

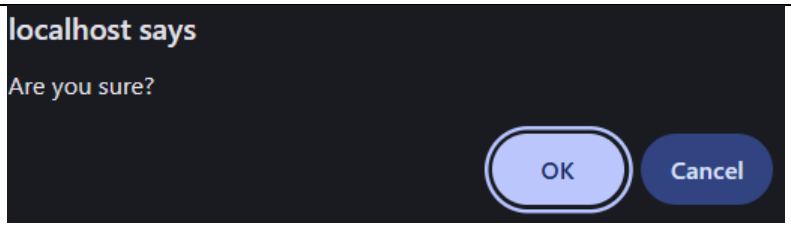
```php
<input type="hidden" name="old_image_path" value="<?= $product['image_path'] ?>">
```

Current Image

Choose File No file chosen

Update Product Cancel

This line in edit_product use existing data to pre-fill product image



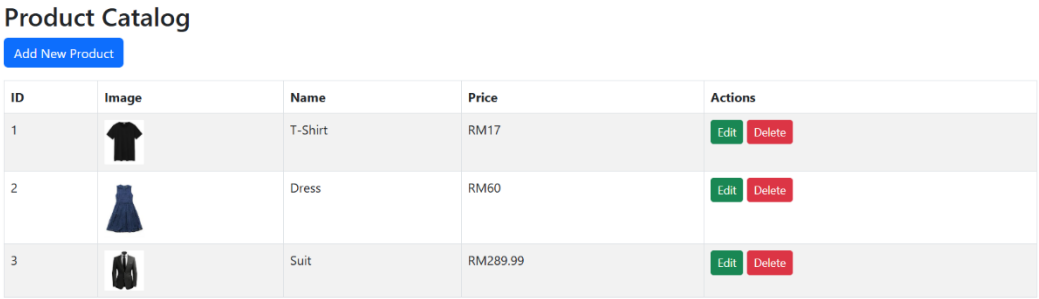
JS Confirmation is use when we wanted to delete a product

Code Structure & Technical Accuracy	Consistently organizes, constructs, and executes well-structured, readable, and error-free PHP code.	Code is generally organized and functional with 1-2 errors or readability issues.	Code structure is partially organized but contains more than 3 frequent syntaxs or logical errors.	Code is poorly constructed, disorganized, and unable to execute properly.

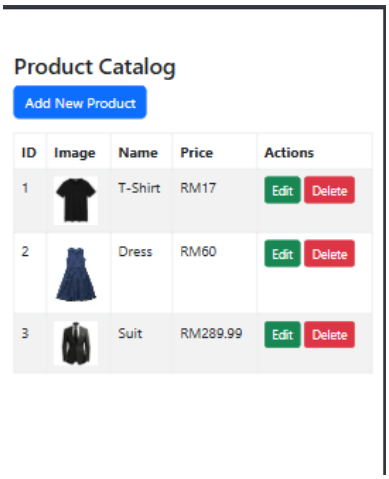
PHP logic is put at the top of the codes, then HTML is put after the PHP logic end

```
<?php include 'db.php'; ?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Product Catalog</title>
```

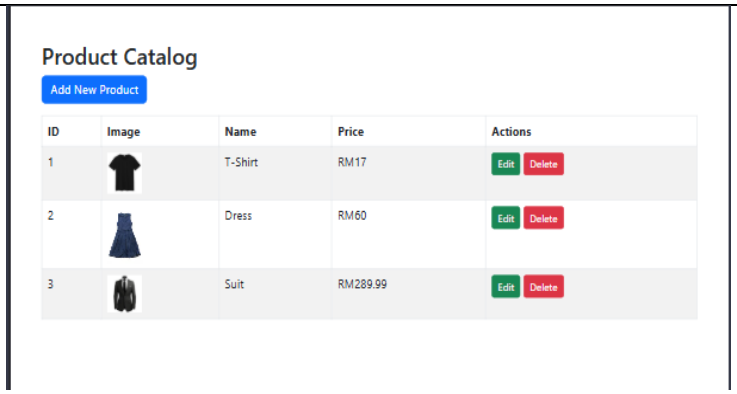
Responsive Design & User Interface Adaptability	Skillfully constructs and applies responsive layout techniques ensuring the application adapts effectively across desktop, tablet, and mobile screens with consistent usability.	Correctly applies responsive design techniques with 1-2 layout issues or usability inconsistencies across devices.	Demonstrates limited ability to apply responsive design; more than 3 issues regards layout, adapts partially but usability is inconsistent.	Fails to construct responsive layout; interface is not functional or usable across different screen sizes.



Regular Web View



Iphone 14 Pro Max View



Ipad Pro View